

# Test Plan Document

## Smart Expense & Budget Management System

Name: \_\_\_\_\_ SRN: \_\_\_\_\_ Section: \_\_\_\_\_

### 1. Purpose

This document lists the test cases written for the implemented use cases of the Smart Expense & Budget Management System. It covers unit, integration and system test cases. Each case is run manually and the Actual Result and Test Result columns are filled in after the run.

### 2. Test Environment

|               |                                                                                   |
|---------------|-----------------------------------------------------------------------------------|
| Application   | Smart Expense & Budget Management System                                          |
| Testing type  | Manual testing                                                                    |
| Browsers      | Chrome and Firefox, latest versions, on desktop and mobile                        |
| Test accounts | Two separate accounts, used to check that one user cannot see another user's data |
| Test data     | Sample values only. No real financial data is used in testing.                    |

### 3. Column Meanings

|                       |                                                                                          |
|-----------------------|------------------------------------------------------------------------------------------|
| Test Case ID          | Unique ID for each test case. UT = unit test, IT = integration test, ST = system test.   |
| Name of the module    | The main module or sub module being tested.                                              |
| Test Case Description | Short summary of what the test is checking.                                              |
| Pre-Conditions        | Anything that must be done or must exist before the test can run.                        |
| Test Steps            | The steps to follow, in the order they are to be carried out.                            |
| Test Data             | The input used for the test. Both valid and invalid values are covered across the cases. |
| Expected Results      | What should happen, including the exact message shown on screen.                         |
| Actual Result         | Filled in after the test is run.                                                         |
| Test Result           | Pass if the actual result matches the expected result, otherwise Fail.                   |

### 4. Test Cases

#### 1. User Registration & Login (UC-01)

| Test Case ID | Name of Module            | Test case description                | Pre-conditions                                 | Test Steps                                                                         | Test data                                                       | Expected Results                                          | Actual Result | Test Result |
|--------------|---------------------------|--------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------|---------------|-------------|
| UT_01        | User Registration & Login | Test registration with valid details | App is open in Chrome on the registration page | 1: Open the sign up page<br>2: Enter name, email and password<br>3: Click Register | Name: Test User<br>Email: user1@test.com<br>Password: Test@1234 | Account is created and the user is taken to the dashboard |               |             |

| Test Case ID | Name of Module            | Test case description                                               | Pre-conditions                                 | Test Steps                                                                                             | Test data                                    | Expected Results                                                      | Actual Result | Test Result |
|--------------|---------------------------|---------------------------------------------------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------|-----------------------------------------------------------------------|---------------|-------------|
| UT_02        | User Registration & Login | Test login with a valid account                                     | A registered account already exists            | 1: Open the login page<br>2: Enter email and password<br>3: Click Login                                | Email: user1@test.com<br>Password: Test@1234 | Login succeeds and the dashboard is displayed                         |               |             |
| UT_03        | User Registration & Login | Test login with a wrong password                                    | A registered account already exists            | 1: Open the login page<br>2: Enter correct email with a wrong password<br>3: Click Login               | Email: user1@test.com<br>Password: Wrong@999 | Login is refused and the message "Invalid email or password" is shown |               |             |
| UT_04        | User Registration & Login | Test registration with a duplicate email                            | user1@test.com is already registered           | 1: Open the sign up page<br>2: Enter the same email again<br>3: Click Register                         | Email: user1@test.com                        | Registration is refused with the message "Email already registered"   |               |             |
| UT_05        | User Registration & Login | Test that required fields are validated                             | App is on the registration page                | 1: Leave the email and password blank<br>2: Click Register                                             | Email: (blank)<br>Password: (blank)          | Form is not submitted and both fields are marked as required          |               |             |
| UT_06        | User Registration & Login | Check that the password is not stored in plain text (SR02)          | One account has been registered                | 1: Register a new account<br>2: Open the users table in the database<br>3: Look at the password column | Password: Test@1234                          | The stored value is a hash, the original password is not readable     |               |             |
| IT_01        | Login + Dashboard         | Check that the dashboard loads the logged-in user's own data        | Two accounts exist, each with its own expenses | 1: Log in as user1<br>2: Note the totals on the dashboard<br>3: Log out and log in as user2            | user1@test.com,<br>user2@test.com            | Each user sees only their own totals, no data from the other account  |               |             |
| IT_02        | Login + Session           | Check that an idle session expires (SR07)                           | User is logged in and the timeout is set       | 1: Log in<br>2: Leave the app idle past the timeout<br>3: Click any menu item                          | Idle time: just over the set timeout         | The session ends and the user is sent back to the login page          |               |             |
| ST_01        | End to end                | Check that a user cannot reach data pages without logging in (SR01) | No active session in the browser               | 1: Clear cookies<br>2: Paste the dashboard URL directly in the address bar                             | URL: /dashboard                              | The app redirects to login and shows no financial data                |               |             |

## 2. Income & Expense Records (UC-02, UC-03)

| Test Case ID | Name of Module           | Test case description                         | Pre-conditions                                     | Test Steps                                                                             | Test data                                    | Expected Results                                                          | Actual Result | Test Result |
|--------------|--------------------------|-----------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------|---------------------------------------------------------------------------|---------------|-------------|
| UT_07        | Income & Expense Records | Test adding a valid expense                   | User is logged in and at least one category exists | 1: Click Add Expense<br>2: Enter amount, date and category<br>3: Click Save            | Amount: 450<br>Date: today<br>Category: Food | Expense is saved and appears at the top of the expense list               |               |             |
| UT_08        | Income & Expense Records | Test adding an expense with a negative amount | User is on the Add Expense form                    | 1: Enter a negative amount<br>2: Click Save                                            | Amount: -200                                 | Entry is refused with the message "Amount must be greater than zero"      |               |             |
| UT_09        | Income & Expense Records | Test adding an expense with a future date     | User is on the Add Expense form                    | 1: Pick a date after today<br>2: Click Save                                            | Date: today + 10 days                        | Entry is refused with the message "Date cannot be in the future"          |               |             |
| UT_10        | Income & Expense Records | Test editing an existing expense              | At least one expense already exists                | 1: Open the expense list<br>2: Click Edit on an entry<br>3: Change the amount and save | Old amount: 450<br>New amount: 600           | The updated amount is shown in the list and the totals change accordingly |               |             |

| Test Case ID | Name of Module           | Test case description                                     | Pre-conditions                                           | Test Steps                                                                                         | Test data                              | Expected Results                                                         | Actual Result | Test Result |
|--------------|--------------------------|-----------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------|----------------------------------------|--------------------------------------------------------------------------|---------------|-------------|
| UT_11        | Income & Expense Records | Test deleting an expense                                  | At least one expense already exists                      | 1: Open the expense list<br>2: Click Delete<br>3: Confirm                                          | Expense: 600, Food                     | Entry is removed from the list and no longer counted in the totals       |               |             |
| UT_12        | Income & Expense Records | Test adding an income record                              | User is logged in                                        | 1: Click Add Income<br>2: Enter amount and source<br>3: Click Save                                 | Amount: 15000<br>Source: Part-time job | Income is saved and the available balance goes up by the same amount     |               |             |
| UT_13        | Income & Expense Records | Test the category suggestion (FR05)                       | The suggestion feature is switched on                    | 1: Open Add Expense<br>2: Type a merchant name in the note field                                   | Note: Metro Station                    | A category such as Travel is suggested, and the user can still change it |               |             |
| IT_03        | Expense + Budget         | Check that adding an expense updates the remaining budget | A monthly budget of 10000 is set with 2000 already spent | 1: Note the remaining budget<br>2: Add an expense of 500<br>3: Look at the remaining budget again  | Expense: 500                           | Remaining budget drops from 8000 to 7500 without needing a page refresh  |               |             |
| IT_04        | Expense + Charts         | Check that a new expense shows up in the charts           | The dashboard charts are already loaded                  | 1: Note the Food slice in the pie chart<br>2: Add an expense under Food<br>3: Reload the dashboard | Expense: 300, Food                     | The Food slice grows and the chart total matches the expense list        |               |             |
| ST_02        | End to end               | Check a user cannot open another user's expense (SR04)    | Two accounts exist, each with expenses                   | 1: Log in as user1<br>2: Note an expense ID belonging to user2<br>3: Request that ID directly      | URL: /expense/{user2 id}               | The request is refused and no data from the other account is returned    |               |             |

### 3. Budget Management & Alerts (UC-04)

| Test Case ID | Name of Module    | Test case description                                      | Pre-conditions                                                  | Test Steps                                                         | Test data                              | Expected Results                                                            | Actual Result | Test Result |
|--------------|-------------------|------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------|-----------------------------------------------------------------------------|---------------|-------------|
| UT_14        | Budget Management | Test setting a monthly budget                              | User is logged in                                               | 1: Open Budgets<br>2: Enter a monthly limit<br>3: Click Save       | Monthly budget: 10000                  | The budget is saved and shown on the dashboard for the current month        |               |             |
| UT_15        | Budget Management | Test setting a zero or negative budget                     | User is on the Budgets page                                     | 1: Enter 0 as the budget<br>2: Click Save                          | Budget: 0                              | The value is refused with a message asking for an amount above zero         |               |             |
| UT_16        | Budget Management | Test setting a category-wise budget                        | A monthly budget already exists                                 | 1: Open Budgets<br>2: Set a limit for the Food category<br>3: Save | Food budget: 3000                      | The Food limit is saved and tracked separately from the monthly budget      |               |             |
| UT_17        | Budget Management | Test the remaining budget calculation (FR08)               | Budget 10000 with expenses of 2500 recorded                     | 1: Open the dashboard<br>2: Read the remaining budget figure       | Budget: 10000<br>Spent: 2500           | Remaining budget shows 7500                                                 |               |             |
| IT_05        | Budget + Alerts   | Check that the warning alert fires at the threshold (FR09) | Budget 10000 with 7500 already spent and the warning set at 80% | 1: Add an expense of 600<br>2: Watch for an alert                  | Expense: 600 (takes the total to 8100) | A warning alert is shown saying the user is close to the budget limit       |               |             |
| IT_06        | Budget + Alerts   | Check the alert when the budget is crossed                 | Budget 10000 with 9800 already spent                            | 1: Add an expense of 500<br>2: Watch for an alert                  | Expense: 500                           | An over-budget alert is shown and the remaining budget is displayed as -300 |               |             |

| Test Case ID | Name of Module        | Test case description                                    | Pre-conditions                                 | Test Steps                                                                                                 | Test data                                  | Expected Results                                                                      | Actual Result | Test Result |
|--------------|-----------------------|----------------------------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------------------------|---------------|-------------|
| IT_07        | Budget + Notification | Check that only one alert is sent per threshold crossing | Budget already crossed once and an alert shown | 1: Add two more small expenses<br>2: Count the alerts received                                             | Expenses: 50, 80                           | No duplicate alerts are sent for the same threshold                                   |               |             |
| ST_03        | End to end            | Full budget cycle from setting a limit to being alerted  | Fresh account with no data                     | 1: Set a monthly budget<br>2: Add expenses until the limit is crossed<br>3: Check the dashboard and alerts | Budget: 5000<br>Expenses: 2000, 2000, 1500 | Remaining budget updates each time and an over-budget alert appears on the last entry |               |             |

#### 4. Analytics, Prediction & Anomaly Detection (UC-05)

| Test Case ID | Name of Module              | Test case description                                           | Pre-conditions                                        | Test Steps                                                                                    | Test data                            | Expected Results                                                                  | Actual Result | Test Result |
|--------------|-----------------------------|-----------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------|---------------|-------------|
| UT_18        | Analytics & Charts          | Test the pie chart totals by category (FR10)                    | Expenses exist across at least three categories       | 1: Open the dashboard<br>2: Compare each pie slice with the expense list                      | Food 1200, Travel 800, Bills 2000    | Each slice matches the category total in the expense list                         |               |             |
| UT_19        | Analytics & Charts          | Test the monthly trend line chart                               | Expenses exist for at least three months              | 1: Open Analytics<br>2: Select the last 3 months                                              | Period: last 3 months                | A line chart is drawn with one point per month and correct monthly totals         |               |             |
| UT_20        | Anomaly Detection           | Test that an unusual expense is flagged (FR11)                  | The user normally spends 200 to 600 on Food           | 1: Add an unusually large Food expense<br>2: Look for a flag on the entry                     | Amount: 9000, Food                   | The entry is flagged for review with a note asking the user to confirm the amount |               |             |
| UT_21        | Anomaly Detection           | Test that a normal expense is not flagged                       | Same history as UT_20                                 | 1: Add a typical Food expense<br>2: Look for a flag                                           | Amount: 350, Food                    | The entry is saved normally with no flag                                          |               |             |
| UT_22        | Prediction                  | Test the monthly expenditure forecast (FR12)                    | At least three months of expense history exist        | 1: Open the dashboard<br>2: Read the forecast for the current month                           | History: 8000, 8500, 9000            | A forecast is shown in a sensible range with the months it is based on            |               |             |
| UT_23        | Prediction                  | Test the forecast for a new account with no history             | A fresh account with under one month of data          | 1: Open the dashboard<br>2: Look at the forecast area                                         | History: 4 days of data              | No forecast is shown, instead a message saying more history is needed             |               |             |
| IT_08        | Prediction + Recommendation | Check the recommended budget stays within income (FR13)         | Income and at least three months of expenses recorded | 1: Open Budgets<br>2: Click Recommend Budget<br>3: Compare the total with the income          | Income: 15000<br>Average spend: 9000 | A category-wise suggestion is shown and the total does not exceed the income      |               |             |
| ST_04        | End to end                  | Check that the health score reacts to a change in habits (FR18) | An account with a few months of steady spending       | 1: Note the current score<br>2: Add a month of heavy overspending<br>3: Check the score again | Extra expenses: 3 x 5000             | The score drops and a short reason for the change is shown                        |               |             |

#### 5. Savings Goals & Recurring Expenses (UC-06, UC-07)

| Test Case ID | Name of Module | Test case description               | Pre-conditions    | Test Steps                                                            | Test data                         | Expected Results                          | Actual Result | Test Result |
|--------------|----------------|-------------------------------------|-------------------|-----------------------------------------------------------------------|-----------------------------------|-------------------------------------------|---------------|-------------|
| UT_24        | Savings Goals  | Test creating a savings goal (FR14) | User is logged in | 1: Open Savings Goals<br>2: Enter a name and target amount<br>3: Save | Goal: New laptop<br>Target: 50000 | The goal is created and shows 0% progress |               |             |

| Test Case ID | Name of Module       | Test case description                                           | Pre-conditions                                      | Test Steps                                                              | Test data                                        | Expected Results                                                                          | Actual Result | Test Result |
|--------------|----------------------|-----------------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------------|---------------|-------------|
| UT_25        | Savings Goals        | Test the time-to-goal estimate (FR15)                           | A goal exists and the user has a steady saving rate | 1: Open the goal<br>2: Read the estimated time                          | Target: 50000<br>Saving rate: 5000 a month       | An estimate of about 10 months is shown                                                   |               |             |
| UT_26        | Savings Goals        | Test the estimate when nothing is being saved                   | A goal exists but the user saved nothing last month | 1: Open the goal<br>2: Read the estimated time                          | Saving rate: 0                                   | No number is shown, instead a message that the goal cannot be reached at the current rate |               |             |
| UT_27        | Recurring Expenses   | Test creating a recurring expense (FR16)                        | User is logged in                                   | 1: Open Recurring<br>2: Enter amount, category and cycle<br>3: Save     | Amount: 6000<br>Category: Rent<br>Cycle: monthly | The recurring expense is saved and listed with its next due date                          |               |             |
| UT_28        | Recurring Expenses   | Test that a recurring expense is not created twice in one cycle | A monthly recurring expense already exists          | 1: Move the system date into the next cycle<br>2: Open the expense list | Cycle: monthly                                   | Exactly one entry is created for the new cycle, not two                                   |               |             |
| IT_09        | Recurring + Reminder | Check the reminder before a due date (FR17)                     | A recurring expense is due in a few days            | 1: Move the date to within the reminder window<br>2: Open the app       | Due in: 3 days                                   | A reminder is shown for the upcoming payment                                              |               |             |
| IT_10        | Recurring + Reminder | Check that reminders stop once the payment is marked paid       | A reminder is currently showing                     | 1: Mark the payment as paid<br>2: Reopen the app                        | Status: paid                                     | The reminder no longer appears for that cycle                                             |               |             |

## 6. Simulator, Reports & Cross-cutting Checks (UC-08, UC-09, UC-10)

| Test Case ID | Name of Module    | Test case description                                  | Pre-conditions                              | Test Steps                                                                | Test data                                | Expected Results                                                                   | Actual Result | Test Result |
|--------------|-------------------|--------------------------------------------------------|---------------------------------------------|---------------------------------------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------|---------------|-------------|
| UT_29        | What-If Simulator | Test simulating an affordable purchase (FR19)          | Budget and expense data exist for the month | 1: Open the simulator<br>2: Enter a planned purchase<br>3: Run            | Purchase: 2000<br>Remaining budget: 7500 | The result shows the budget left after the purchase and marks it as affordable     |               |             |
| UT_30        | What-If Simulator | Test simulating a purchase the user cannot afford      | Same starting data as UT_29                 | 1: Open the simulator<br>2: Enter a large purchase<br>3: Run              | Purchase: 40000                          | The result warns that the purchase goes past the budget and shows the shortfall    |               |             |
| UT_31        | What-If Simulator | Check that a simulation is not saved as a real expense | A simulation has just been run              | 1: Run a simulation<br>2: Open the expense list                           | Purchase: 40000                          | No new expense appears in the list and the remaining budget is unchanged           |               |             |
| UT_32        | Export Reports    | Test exporting a report (FR20)                         | The account has at least one month of data  | 1: Open Reports<br>2: Pick a month<br>3: Click Export                     | Month: last month<br>Format: PDF         | A file downloads with figures matching the dashboard for that month                |               |             |
| UT_33        | Export Reports    | Test exporting a month with no data                    | A month with no expenses is selected        | 1: Open Reports<br>2: Pick an empty month<br>3: Click Export              | Month: with no entries                   | The app shows a message that there is nothing to export instead of an empty file   |               |             |
| IT_11        | Input Validation  | Check that unsafe input is rejected (SR05)             | User is on the Add Expense form             | 1: Enter a script tag in the note field<br>2: Save<br>3: Reopen the entry | Note:<br><script>alert(1)</script>       | The input is rejected or escaped, and nothing is executed when the entry is opened |               |             |

| Test Case ID | Name of Module | Test case description                         | Pre-conditions                   | Test Steps                                                                                                                      | Test data                                                  | Expected Results                                                                   | Actual Result | Test Result |
|--------------|----------------|-----------------------------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------|---------------|-------------|
| ST_05        | End to end     | Full user journey from registration to export | Fresh account                    | 1: Register and log in<br>2: Add income and a few expenses<br>3: Set a budget<br>4: Check the dashboard<br>5: Export the report | Income: 15000<br>Expenses: 500, 1200, 800<br>Budget: 10000 | Every step works in order and the exported figures match what the dashboard showed |               |             |
| ST_06        | End to end     | Check that records survive a restart (NFR10)  | Several expenses have been saved | 1: Note the totals<br>2: Restart the server<br>3: Log in and check again                                                        | Expenses: 5 entries                                        | All entries are still present and the totals are unchanged                         |               |             |

## 5. Summary

| Type of Testing          | Number of Test Cases | Covers                                                           |
|--------------------------|----------------------|------------------------------------------------------------------|
| Unit Testing (UT)        | 33                   | Individual functions and form validations                        |
| Integration Testing (IT) | 11                   | Modules working together, e.g. expense entry updating the budget |
| System Testing (ST)      | 6                    | Complete user journeys and cross-cutting behaviour               |
| <b>Total</b>             | 50                   |                                                                  |

*Note: the Actual Result and Test Result columns are left blank on purpose. They are to be filled in by hand while running each test case, as required by the instructions.*